

RV32I -Based, 5-Stage Pipelined Processor Design Report

1.INTRODUCTION AND DESIGN OBJECTIVES

Within the scope of this project, a 32-bit multi-cycle processor core with a 5-stage pipeline architecture compatible with the RISC-V architecture has been designed and verified. The designed microprocessor supports the RV32I base integer instruction set. In addition, CLZ, CTZ, and CPOP instructions have been hardware-integrated into the ALU.

The core technical justifications for making specific microarchitectural choices in the design are as follows:

- **RISC-V Architecture:** RISC-V has allowed custom bit manipulation instructions required by chip design projects to be integrated into the main hardware due to its extensible structure.
- **5-Stage Pipeline:** In single-cycle designs, the maximum clock frequency (F_{max}) that the processor can achieve is limited by the memory read time, which is the slowest operation in the system. To overcome this constraint, the architecture has been divided into 5 hardware stages, namely IF, ID, EX, MEM, and WB, in accordance with classic Patterson & Hennessy principles. Through this parallelization, it is targeted for the CPI value to approach the ideal.
- **Hardware Data Forwarding:** In pipeline architectures, interdependent consecutive instructions cause the pipeline to stall. To prevent this drop in throughput, data forwarding logic has been incorporated into the design, allowing current data that has not yet reached the writeback stage to be routed directly to the input of the execution stage.

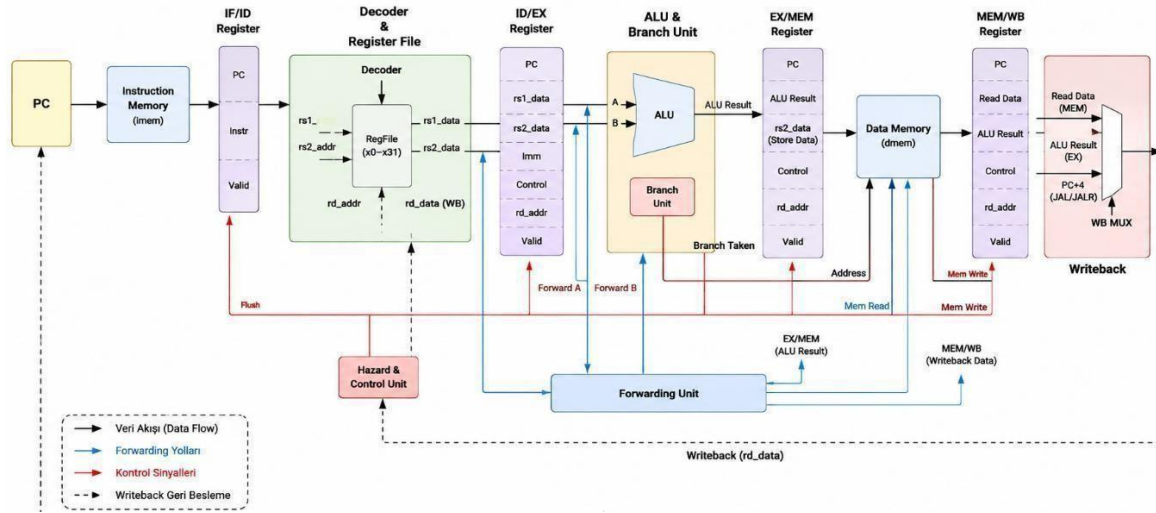
REQUIREMENT ANALYSIS AND SPECIFICATIONS

The hardware realization status of the design specifications and requirements determined at the beginning of the project is summarized in the table. All specified targets have been designed at the RTL level and coded using SystemVerilog.

Required Design / Feature	Check	Description / Hardware Implementation
RV32I Instruction Set	✓	All requested basic arithmetic, logical, and memory instructions are supported.
CLZ, CTZ, CPOP	✓	Integrated into the ALU module as a hardware implementation.
5-Stage Pipeline	✓	The IF, ID, EX, MEM, and WB stages operate actively in parallel within the architecture.
Data Forwarding (Bypass)	✓	Data hazards have been resolved via EX->EX and MEM->EX forwarding paths.
Hazard Management (Stall)	✓	A 1-cycle stall is inserted into the pipeline during Load-Use scenarios.
Control Hazard (Flush)	✓	The IF and ID registers are hardware-flushed upon a branch.
Variable-Latency Memory	✓	The pipeline is frozen with an asynchronous lock until the memory responds.
Commit Log Interface	✓	Every completed instruction is transmitted to the external testbench environment via the <code>commit_o</code> structure.

THEORETICAL BACKGROUND AND SYSTEM ARCHITECTURE

The designed RV32I processor architecture has been divided into 5 independent stages, referencing the classic Patterson & Hennessy microarchitecture, in order to achieve high clock frequencies and increase instruction throughput.



The general hardware architecture and data flow paths of the processor are illustrated.

1.1. Pipeline Stages

The processor datapath consists of the following 5 hardware-isolated stages:

1. Instruction Fetch (IF): The 32-bit instruction at the memory address pointed to by the program counter is fetched via the Instruction Memory.
2. Instruction Decode (ID): The fetched instruction is transferred to the riscv_decoder module. Opcode, funct3, and funct7 values are decoded to generate control signals that will manage the other stages of the processor. Simultaneously, source data (rs1, rs2) is read from the Register File.
3. Execution (EX): This is the main calculation layer where the riscv_alu module is located. Arithmetic, logical, and bit manipulation operations are performed here. Branch/jump target addresses and decisions are finalized in hardware at this stage.
4. Memory Access (MEM): A read or write request is sent to the external Data Memory via the load_store_unit.
5. Writeback (WB): The result from the ALU, the data read from memory, or the PC+4 address is synchronously written to the target register.

1.2. Pipeline Registers

The critical data packets carried by the intermediate registers used to ensure synchronization between stages and divide combinational delay are summarized in the table.

Register	Critical Signals and Data Packets Carried
IF/ID	if_id_pc, if_id_instr, if_id_valid
ID/EX	id_ex_pc, id_ex_rs1_data, id_ex_rs2_data, id_ex_imm, Kontrol Sinyalleri
EX/MEM	ex_mem_alu_res, ex_mem_rs2_data, ex_mem_rd_addr, Kontrol Sinyalleri
MEM/WB	mem_wb_alu_res, mem_wb_mem_data, mem_wb_rd_addr, RegWrite

1.3. Submodules and Operational Principles

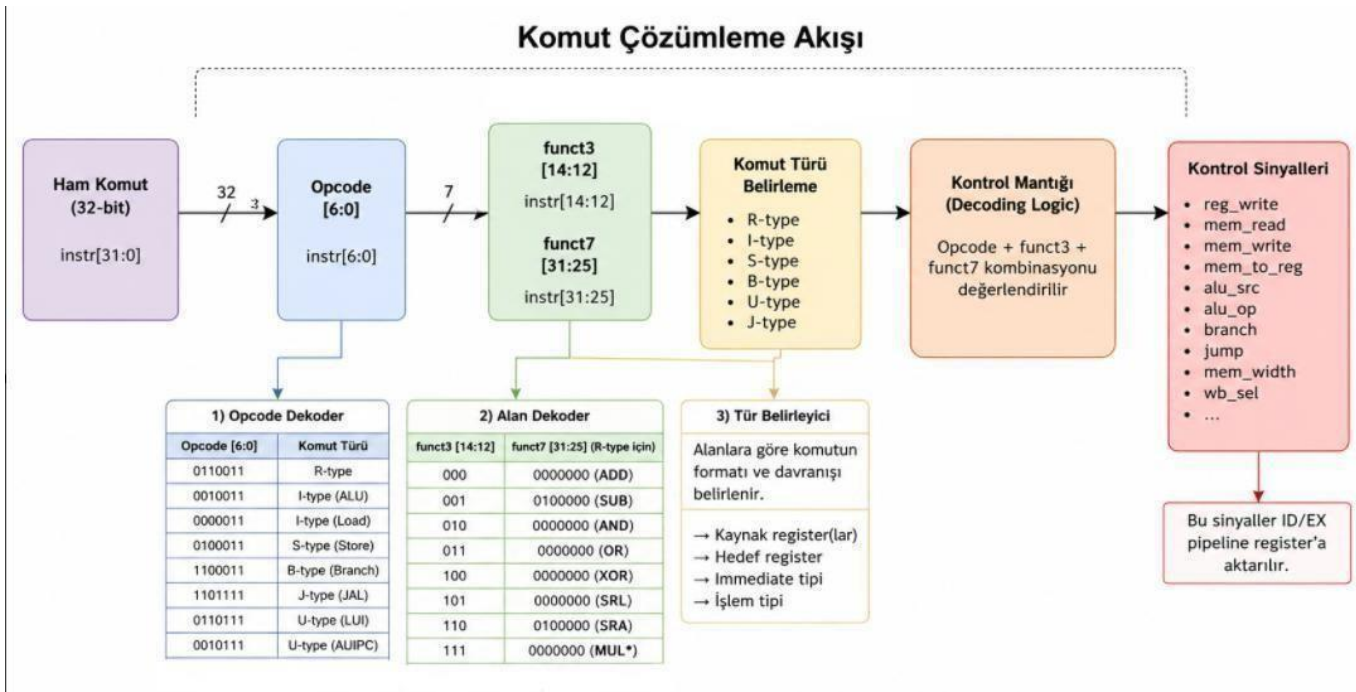
An isolated architecture has been chosen in terms of design sustainability and ease of debugging. The blocks generating control signals and the blocks processing data are physically separated from each other.

1.3.1. Decoder

The Decoder unit breaks down the 32-bit RISC-V instruction to control ALU operations, memory widths (Byte, Half, Word), and immediate value generation. The hardware truth table is provided.

	Opcode (7-bit)	Reg Write	Mem Read	Mem Write	Branch	Jump	ALU_Src_B	WB_Sel
R-Type	0110011	1	0	0	0	0	0 (Reg)	WB_ALU
I-Type	0010011	1	0	0	0	0	1 (Imm)	WB_ALU
Load	0000011	1	1	0	0	0	1 (Imm)	WB_MEM
Store	0100011	0	0	1	0	0	1 (Imm)	X
Branch	1100011	0	0	0	1	0	0 (Reg)	X
JAL	1101111	1	0	0	0	1	X	WB_PC4
JALR	1100111	1	0	0	0	1	1 (Imm)	WB_PC4
LUI	0110111	1	0	0	0	0	1 (Imm)	WB_ALU

The hardware decoding process occurs in a combinational flow rather than sequentially to prevent latency, structured as follows:



1.3.2. ALU and Custom Instructions

In addition to basic RV32I operations, the ALU module also supports non-standard bit manipulation instructions (CLZ, CTZ, CPOP) with dedicated flags. These instructions are calculated in a single cycle using synthesizable hardware scanners instead of software loops.

Control Signal	Hardware Operation / Function	Description
ALU_ADD / SUB	Addition / Subtraction	Standard arithmetic operations.
ALU_AND / OR / XOR	Logical Operations	Bitwise logical operators.
ALU_SLL / SRL / SRA	Shift	Logical and arithmetic bit shift operations.
ALU_SLT / SLTU	Comparison	Signed and unsigned less-than checks.
ALU_CLZ	Count Leading Zeros	Counting consecutive zeros from the most significant bit (MSB) side.
ALU_CTZ	Count Trailing Zeros	Counting consecutive zeros from the least significant bit (LSB) side.
ALU_CPOP	Count Population	Counting the total number of "1" bits within a 32-bit data word.

1.3.3. Branch Unit

Branch conditions have been designed as an independent hardware module named `branch_unit.sv`, without relying on the subtraction result of the ALU. Equal, Less Than, and Unsigned Less Than conditions are calculated in the Execution stage and transferred directly to the Hazard Unit module. Comparison operations (`br_eq`, `br_lt`, `br_ltu`) are pre-calculated using parallel logic blocks independently of the ALU's subtraction result. This architectural choice minimizes the critical path delay, thereby increasing the maximum operating frequency (F_{max}) of the processor. In addition, if the branch conditions are met, the calculated target address is routed to the PC multiplexer without delay, while simultaneously triggering the hardware flush signal via the Hazard Unit to clear the IF and ID stages.

4.1. Implementation of Bit Manipulation Instructions Within the ALU

The CLZ, CTZ, and CPOP instructions, which are not present in the standard RV32I instruction set, are decoded within the decoder via the `decode_bitmanip()` function and implemented in hardware using synthesizable for-loops inside `riscv_alu.sv`. Consequently, the added instructions are naturally integrated into the existing ALU control structure.

ALU_CLZ: begin

```
alu_result_o = 32; // Varsayılan: Tümü sıfır
for (int i = 31; i >= 0; i--) begin
    if (alu_in_a_i[i]) begin
        alu_result_o = 31 - i;
        break; // İlk '1' görüldüğünde donanımsal tarama durur
    end
end
end
```

The synthesis tool translates the above for-loop into hardware not as a time-consuming sequential software loop, but as a 32-input MUX and Comparator Tree. Similarly, the CPOP operation is synthesized by iterating across all bits to produce parallel adders.

4.2. Byte Lane Alignment in the Load/Store Unit

Memory read and write operations are executed via the external data memory provided by the testbench. The `load_store_unit.sv` module is responsible for formatting the raw data coming from the external memory (`mem_read_data_i`) according to the instruction's data width (Byte, Half, Word) and signed/unsigned status.

In Store operations, the generated raw data is transmitted directly to the memory interface, and the placement into the corresponding byte lane is handled by the external memory module in accordance with the hardware interface protocol. In Read operations, however, the targeted byte or half-word is isolated from the aligned data coming from memory and transferred to the pipeline in 32-bit format via hardware multiplexers:

```
assign load_byte = mem_read_data_i[7:0];
assign load_half = mem_read_data_i[15:0];

always_comb begin
    mem_read_data_formatted_o = mem_read_data_i;

    if (ex_mem_mem_read_i) begin
        unique case (ex_mem_mem_width_i)
            BYTE: mem_read_data_formatted_o = {{24{load_byte[7]}}, load_byte};
            HALF: mem_read_data_formatted_o = {{16{load_half[15]}}, load_half};
            BYTE_U: mem_read_data_formatted_o = {24'b0, load_byte};
            HALF_U: mem_read_data_formatted_o = {16'b0, load_half};
            WORD: mem_read_data_formatted_o = mem_read_data_i;
            default: mem_read_data_formatted_o = mem_read_data_i;
        endcase
    end
end
```

4.3. Data Forwarding and Priority Logic

Designed to resolve data hazards, the `forward_unit.sv` module forwards the latest data from the EX/MEM and MEM/WB stages to the ALU inputs. If there is a write request to the same destination register in both stages, the architecture prioritizes the data in the EX/MEM stage. Furthermore, by adding the `!ex_mem_mem_read_i` condition, incorrect forwarding is prevented during Load-Use hazards where the data has not yet returned from memory.

```
if(ex_mem_reg_write_i && !ex_mem_mem_read_i && (ex_mem_rd_addr_i != 0) &&
(ex_mem_rd_addr_i == id_ex_rsl_addr_i) && ex_mem_valid_i) begin

    forward_a_o = 2'b10; // En güncel veri (EX/MEM aşamasından)

end else if(mem_wb_reg_write_i && (mem_wb_rd_addr_i != 0) && (mem_wb_rd_addr_i ==
id_ex_rsl_addr_i) && mem_wb_valid_i) begin

    forward_a_o = 2'b01; // Bir önceki güncel veri (MEM/WB aşamasından)

end
```

The hardware selection logic of the module is based on the following simple and fixed-priority rule set:

forward_a / b Signal	Data Source	Status / Description
2'b00	Register File	No dependency, normal flow.
2'b01	MEM/WB Register	Data dependency with one instruction gap (MEM to EX).
2'b10	EX/MEM Register	Consecutive instruction dependency (EX to EX / Newest Data).

4.4. Control and Data Hazard Management

Pipeline stalling and flushing operations are managed by `hazard_unit.sv`. The `mem_busy` signal, used by the processor during variable-latency memory accesses, is directly connected to the stall logic as an asynchronous lock.

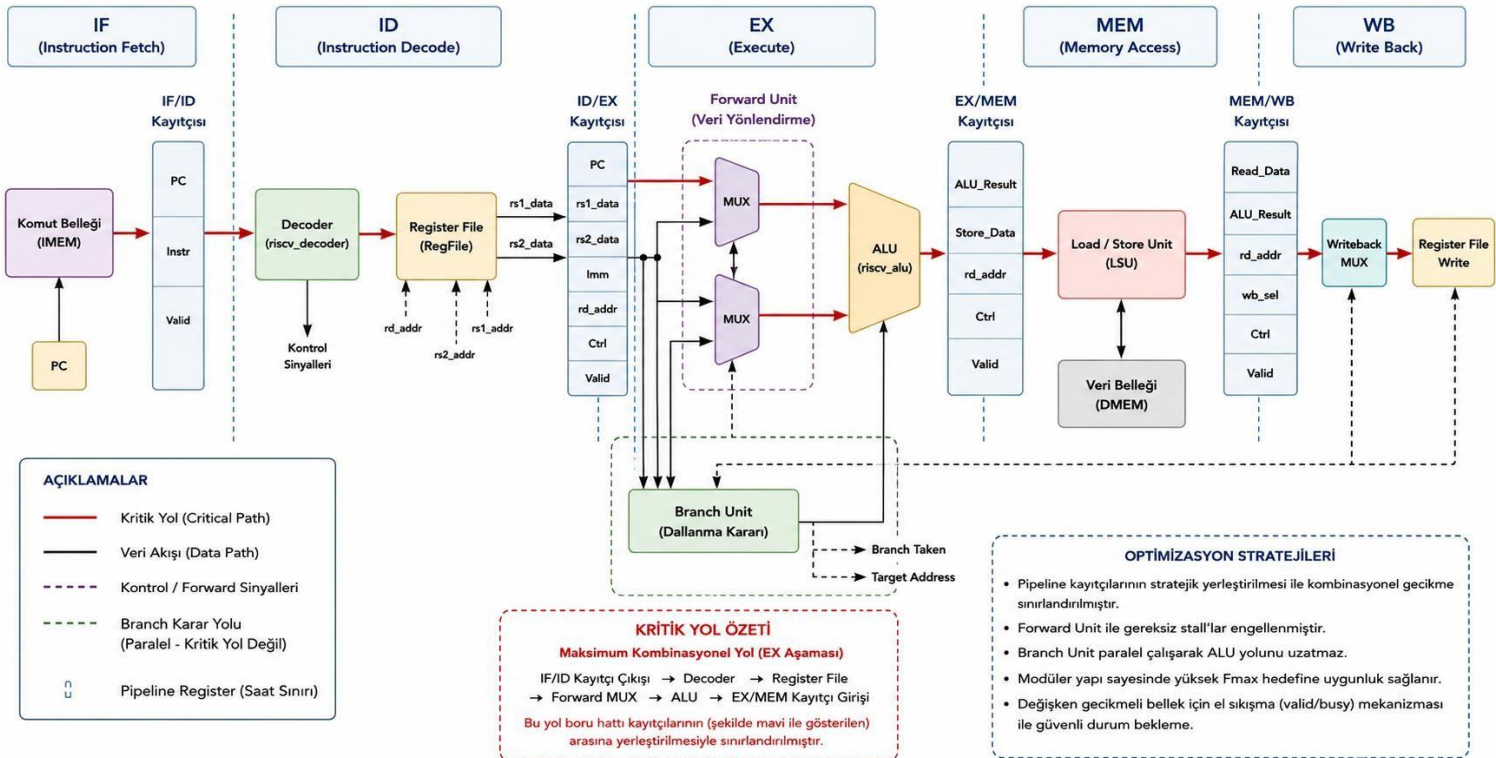
```
assign stall_pipeline_o = mem_stall_i;
assign flush_pipeline_o = control_hazard && !mem_stall_i;
assign load_use_stall_o = data_hazard && !mem_stall_i;
```

To manage asynchronous wait states in external memory scenarios, the pipeline is stalled until the `dmem_res_i.valid` signal is returned from the testbench. In the synthesized circuit, the `mem_stall_i` signal has the highest priority, overriding all other hazard signals; consequently, the processor can neither flush nor advance the pipeline without receiving the memory response. Processor-wide hazard conditions and hardware responses are managed according to the matrix below:

Hazard Condition	Trigger Cause	Hardware Action
Load-Use Hazard	Use of the read data in the immediately following instruction	Stall (Freeze the pipeline for 1 cycle)
Control Hazard	Occurrence of a conditional branch or jump	Flush (Clear incorrectly fetched instructions)
Memory Stall	External memory not yet returning a valid signal	Freeze (Lock down the entire system in hardware)

5. Rationale for Design Decisions

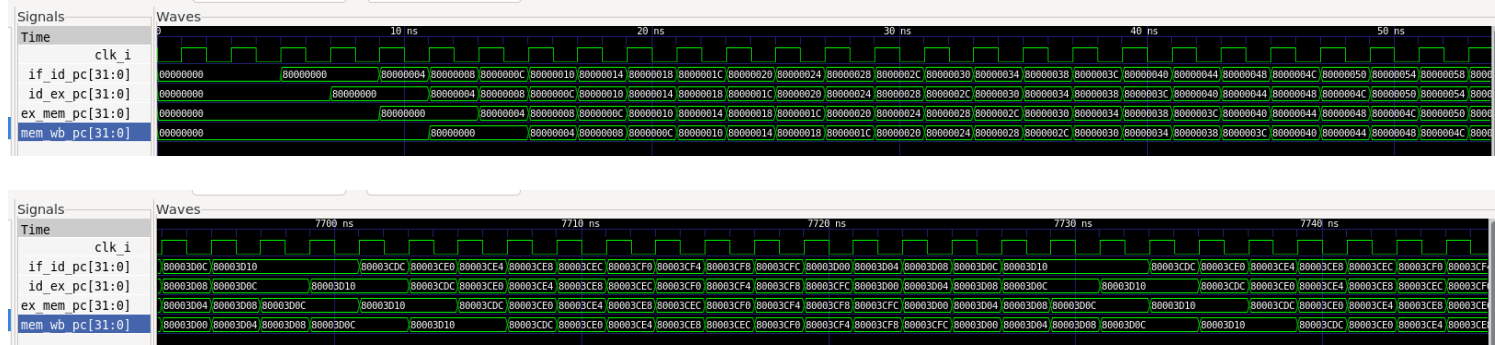
1. **Multi-Cycle and Pipelined Architecture:** In single-cycle processor architectures, the maximum attainable clock frequency ($f_{\max}$) of the system is limited by the memory access time, which represents the operation with the longest delay in the instruction set. To overcome this critical path bottleneck, the hardware is partitioned into 5 stages: IF, ID, EX, MEM, and WB. Pipelining logic enables parallel operation of hardware blocks, optimizing instruction throughput.
2. **Hardware Data Forwarding:** Stalling the pipeline every time data dependencies occur between consecutive instructions in pipelined architectures degrades overall performance. By bypassing the Register File and forwarding data directly from the EX/MEM and MEM/WB registers to the inputs of the execution unit, data hazard latency is minimized, and unnecessary stall cycles are prevented.
3. **Direct Memory Interface and Asynchronous Wait Logic:** To tolerate asynchronous wait states in external memory scenarios, a combinational lock structure (mem_busy) tied to the valid signal from the external memory was implemented. In alignment with the testbench interface, the system hardware-stalls the pipeline until the dmem_res_i.valid signal is asserted. Data read/write operations and byte lane alignments are handled directly within the input-output boundaries provided by the testbench.
4. **Critical Path Optimization and Synthesizability:** From a hardware synthesis perspective, the longest combinational delay determining the processor's maximum operating frequency concentrates in the EX stage of the proposed architecture. Generating branch decisions within the Branch Unit via parallel comparators independent of the ALU prevents the control logic from extending the ALU data path, thereby reducing critical path delay. Physically isolating the processor stages with pipeline registers ensures that the design can satisfy timing requirements in ASIC/FPGA synthesis tools.



6. Verification and Waveform Analysis

The functional correctness of the processor architecture was tested in a Verilator-based simulation environment, and the timing behavior of the design was analyzed using the GTKWave tool. During the tests, scenarios involving memory access, data hazards, and control hazards were executed alongside the RV32I instruction set. The waveform analyses below verify that pipeline synchronization, the data forwarding mechanism, and hazard management operate as intended at the RTL level..

6.1. Continuous Pipeline Flow

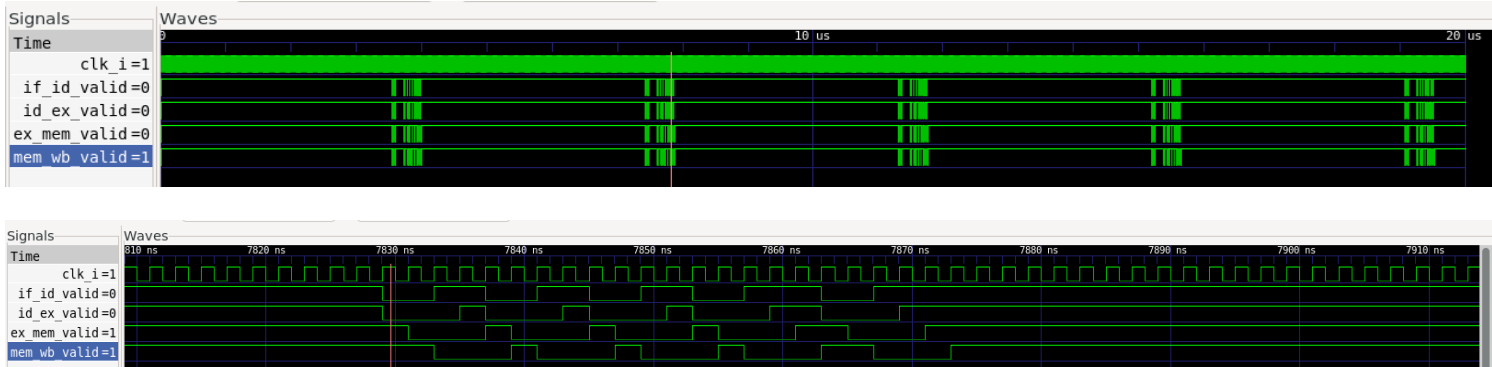


When the if_id_pc, id_ex_pc, ex_mem_pc, and mem_wb_pc signals are examined simultaneously on the waveform, it is clearly observed that different instructions reside in different stages on each clock cycle.

Ideal Pipeline Flow Matrix

Clock	IF Stage PC	ID Stage PC	EX Stage PC	MEM Stage PC	WB Stage PC
t₀	PC = n				
t₁	PC = n+4	PC = n			
t₂	PC = n+8	PC = n+4	PC = n		
t₃	PC = n+12	PC = n+8	PC = n+4	PC = n	
t₄	PC = n+16	PC = n+12	PC = n+8	PC = n+4	PC = n

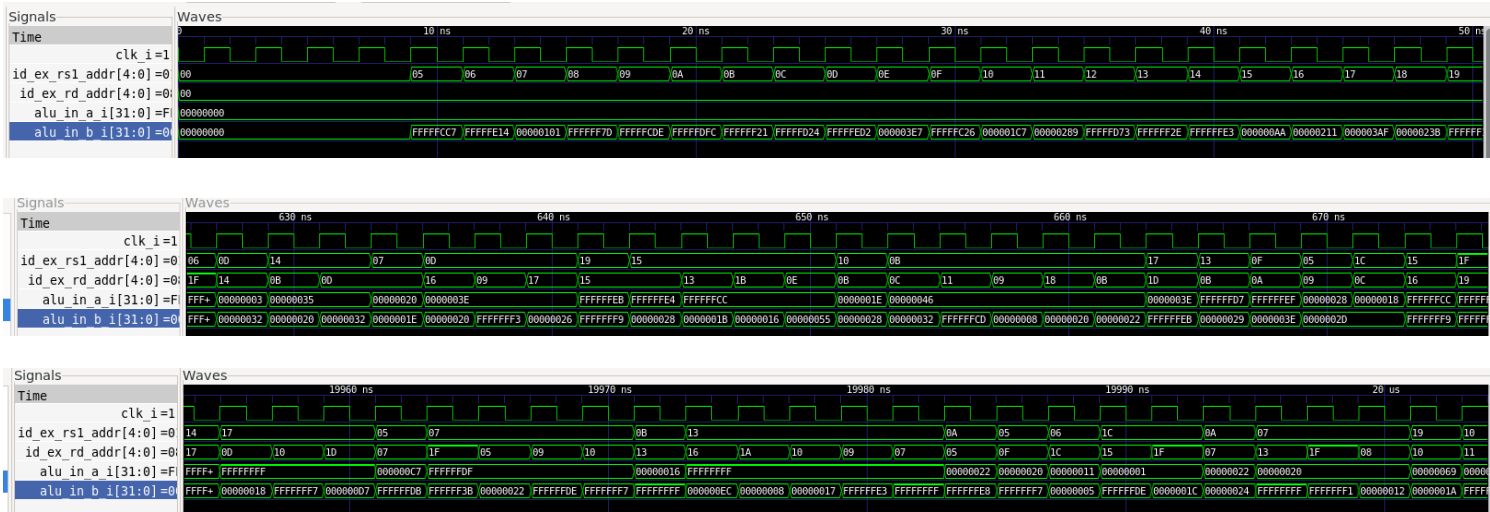
6.2. Pipeline Register Synchronization



The validity bits propagated through the IF/ID, ID/EX, EX/MEM, and MEM/WB registers have been analyzed within the design. As observed in the waveforms, all pipeline registers are updated simultaneously exclusively on the positive edge of the common clock signal. By having the synthesis tool map these structures into standard D-type flip-flop arrays in hardware, combinatorial feedback loops are prevented at the hardware level, the critical path length is reduced, and each stage processes only its dedicated data in an isolated manner.

Additionally, propagating a stage-specific valid bit throughout the pipeline establishes a critical safety layer for hazard management. Whenever a hardware stall is introduced or a branch is taken, the valid bit of the corresponding register is immediately cleared. This hardware-level mechanism prevents canceled or unready instructions from corrupting the architectural state and permanently writing to the Register File or Data Memory. This architecture serves as the RTL-level implementation of the classical Patterson & Hennessy pipelined processor model.

6.3. Data Hazard and EX → EX Forwarding

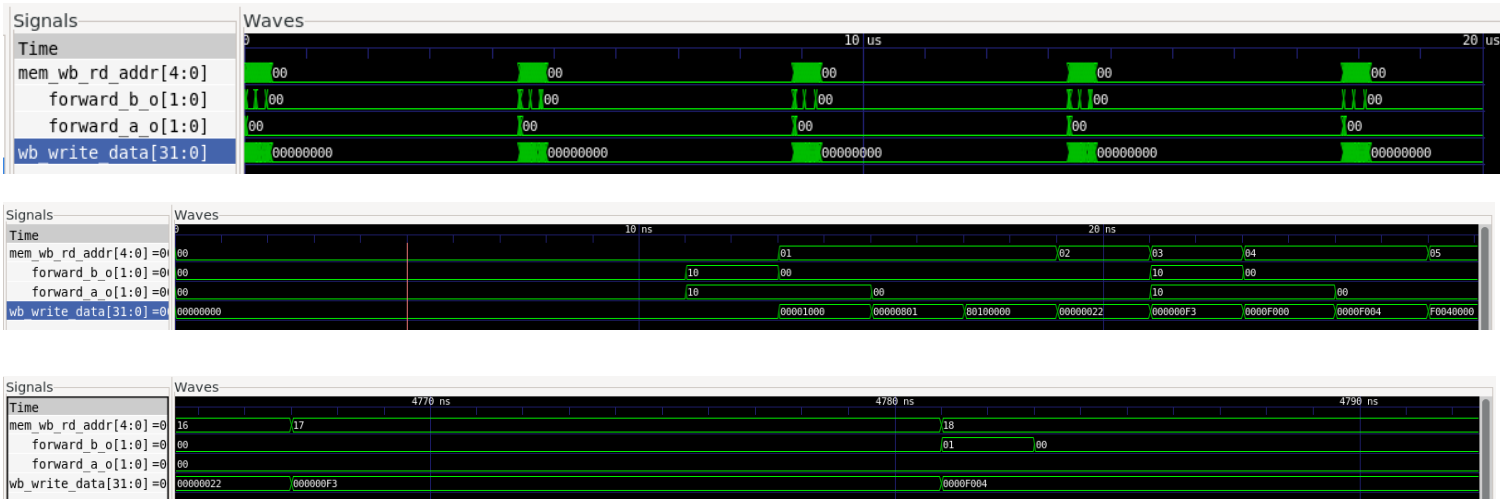


The following instruction sequence creates a direct data dependency (Read-After-Write):

1. ADD x5, x1, x2
2. SUB x6, x5, x3

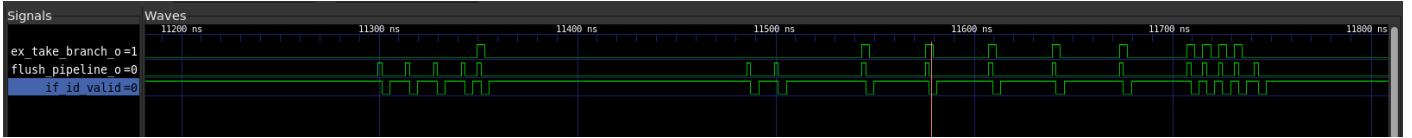
The value of x5 required by the second instruction has not yet been written to the Register File. Under normal circumstances, the pipeline would be expected to stall. However, the waveform shows that as soon as `forward_a = 2'b10`, the operand at the ALU input (`alu_a_i`) is obtained directly from the current ALU result in the EX/MEM register (`ex_mem_alu_res`) rather than the Register File. Consequently, the pipeline does not stall, no bubbles are introduced, and the data is bypassed directly.

6.4. MEM → EX Forwarding



In some scenarios, the required up-to-date data resides not in the EX stage, but one step further in the MEM/WB stage. When the Forwarding Unit detects this condition, it asserts the `forward = 2'b01` signal, routing the data directly from the MEM/WB register to the ALU input. Consequently, consecutive dependencies separated by a single instruction gap can be resolved with zero latency.

6.6. Branch Hazard ve Flush



The target outcomes of conditional branch instructions are finalized in the EX stage. If the branch is taken, the instructions currently residing in the IF and ID stages become incorrect instructions. The waveform shows that in the cycle where `ex_take_branch = 1`, the hardware instantaneously asserts `flush_pipeline_o = 1`. This signal immediately clears the validity bits of the IF/ID and ID/EX registers, thereby preventing the architecture from executing incorrect instructions at the hardware level.

6.7. Variable-Latency Memory Wait State



In the updated testbench infrastructure, the memory delay was set to 5 clock cycles (`dmem_delay = 5`) to test the hardware. The resulting waveforms illustrate the system's hardware response to this delay as follows:

- **Locking:** When the processor begins processing a memory instruction (`is_mem_op = 1`), the `mem_busy` signal is asserted immediately because the memory has not yet responded (`valid = 0`).
- **Waiting:** Throughout the 5 clock cycles that the `mem_busy` signal remains active, `pc_reg` and the pipeline registers are frozen, placing the system into a secure wait state. During this period, the `dmem_req_sent` flag is used to prevent redundant requests from being sent to the testbench.
- **Resuming:** At the end of the cycle when the `valid = 1` acknowledgment signal arrives from memory, `mem_busy` is combinational cleared, and the pipeline resumes its normal flow without any data loss.

This combinational lock structure adapts to asynchronous memory delays at the hardware level without the need for complex state machines.

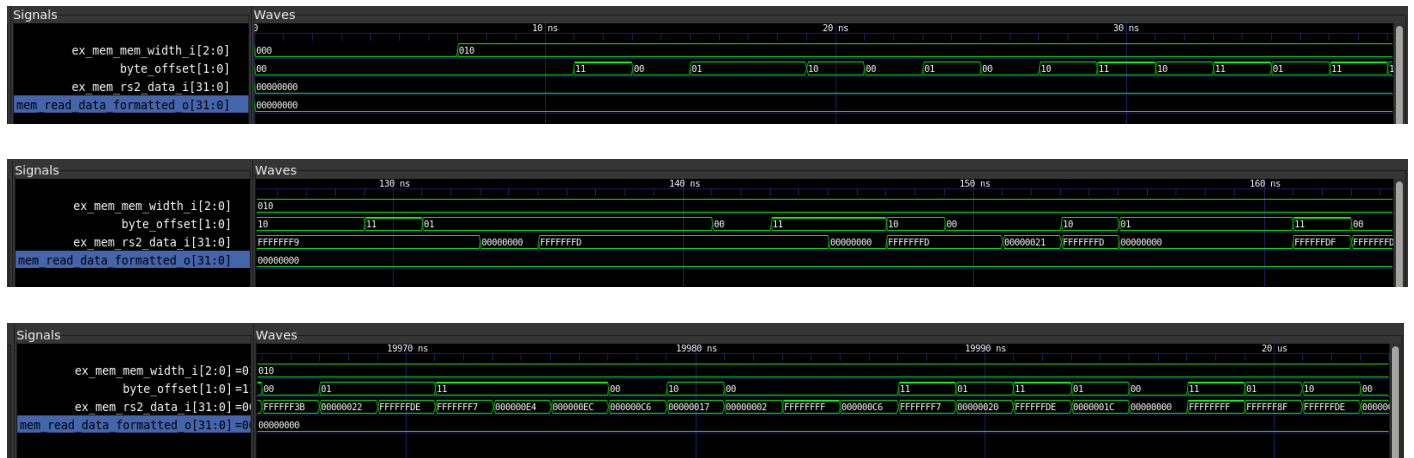
6.8. Writeback Stage and Testbench Log Verification



Instructions completed by the processor are transferred to the Register File or memory interface during the Writeback stage. The waveform above clearly shows how the mem_wb_valid, mem_wb_pc, mem_wb_instr, and final data wb_write_data signals are generated synchronously with the clock pulses.

The testbench environment monitors the write operations occurring in this stage and generates the model.log file. The model.log file obtained at the end of the simulation was compared against the reference model using a Python script, verifying that all instructions produced correct results.

6.9. Hardware Data Alignment Analysis



Since the external memory operates asynchronously and supports various data widths, data read from memory must be transferred to the processor pipeline in a suitable format. When examining the read operations on the waveform, it is observed that the processor evaluates the data coming from the external memory interface according to the data width and signed/unsigned attributes of the respective instruction. The LSU hardware isolates the target byte or half-word from the incoming data, converts it into a 32-bit format via hardware multiplexers, and transfers it to the mem_read_data_formatted_o output. This hardware interface eliminates the need for software masking, ensuring that data reaches the pipeline with the correct size and sign extension.

7. TEST RESULTS AND PERFORMANCE EVALUATION

During the simulation, while processing the machine codes read from the imem.mem file, the processor logs the result of each instruction completed in the WB stage to the model.log file. This recorded structure;

- PC
- Instruction
- Register Write
- Memory Write

Base Test Verification Results

	Total Tested Instructions	Succes (PASS)	Success Rate
R-Type	1275	1275	%100
I-ALU	641	641	%100
Branch	15	15	%100
Jump	7	7	%100
Upper	1	1	%100
Total	1939	1939	%100

TOTAL: 1939/1939 (%100) – All tests passed!

```
burak@DESKTOP-NS93M65:/mnt/c/Users/burak/Desktop/burak_karapınar/Milli_Teknoloji_Uygulama_Projesi$
```

Memory Test Verification Results

	Total Tested Instructions	Succes (PASS)	Success Rate
Load (LW, LH, LHU, LB, LBU)	19	19	%100
Store (SW, SH, SB)	9	9	%100
ALU ve Upper Control	8	8	%100
Total	36	36	%100

TOTAL: 36/36 (%100) – All tests passed!

```
burak@DESKTOP-NS93M65:/mnt/c/Users/burak/Desktop/burak_karapınar/Milli_Teknoloji_Uygulama_Projesi$
```


Cycle Costs by Instruction Classes and CPI Analysis

Instruction Class	Estimated Cycle Consumption	Impact on CPI and Notes
Arithmetic and Logical	1	Ideal case. Stalls are prevented via forwarding (CPI = 1).
Branch - Not Taken	1	If the branch is not taken, the flow continues normally.
Branch - Taken	3	Since the decision is made in the EX stage, IF and ID are cleared.
Jump	3	Until the target is calculated due to the unconditional jump.
Load	1 + N	N: The response latency of the external memory.
Store	1 + N	The processor is frozen with <code>mem_busy</code> until the memory valid signal returns.

8. CONCLUSION AND FUTURE DEVELOPMENTS

In this study, a five-stage pipelined RISC-V processor supporting the RV32I instruction set and additionally capable of executing CLZ, CTZ, and CPOP bit-manipulation instructions was designed and implemented at the RTL level using SystemVerilog. During the design process, the processor was developed as independent modules such as ALU, Decoder, Branch Unit, Forward Unit, Hazard Unit, and Load/Store Unit, with data flow between modules handled via synchronous pipeline registers. To resolve data dependencies, EX→EX and MEM→EX data forwarding mechanisms were implemented; a hardware pipeline flushing mechanism was designed for control hazards, and a valid-based combinational freeze structure was designed for memory accesses.

In the memory subsystem of the design, direct interface integration was implemented to adapt to the asynchronous nature and variable latencies of the external test environment. Variable-latency memory accesses were managed without the need for complex state machines by utilizing `mem_busy`, which operates depending on the valid signal coming from the external memory. Additionally, to ensure data integrity in memory operations, a hardware structure was established within the Load/Store unit to format the pre-aligned raw data coming from the external memory interface according to the instruction's data width and sign status.

As a result of the Verilator-based verification studies performed, the processor achieved 1939/1939 in the Base Test scenario and 36/36 in the Memory Test scenario, producing results compatible with the reference model logs in the testbench environment. Furthermore, thanks to the data forwarding mechanism added to the design, unnecessary stall cycles in the pipeline were reduced, increasing the instruction throughput.

Within the scope of the project, to ensure the functional accuracy of the processor, in addition to waveform analyses, a `commit_o` hardware interface operating synchronously with the external world was integrated. Every instruction emerging from the Writeback stage, along with the program counter and written-back data packets, was transferred via this interface to the Python-based testbench environment and automatically compared against the reference model.

This approach verified that the processor meets the hardware requirements in the design specification without disrupting its architectural state.

- **Dynamic Branch Prediction:** Reduction of flush penalties caused by control hazards,
- **Cache Architecture:** Reduction of memory access latencies via L1 Instruction and Data Cache integration,
- **RV32M Extension:** Expanding the instruction set to the RV32IM level by adding hardware multiplication and division units,
- **Performance Counters:** Hardware measurement of CPI, IPC, and pipeline utilization rates,
- **System Integration:** Integration of the processor into larger system-on-chip architectures by adding industrial memory interfaces,
- **Privileged Modes (Privileged ISA):** Provision of operating system-supported operation modes by adding interrupt, exception, and CSR mechanisms.

APPENDICES

Appendix A: RV32I Instruction Formats

The RISC-V architecture keeps all instructions fixed at a length of 32 bits to reduce hardware complexity and utilizes 6 core instruction formats (R, I, S, B, U, J). The Decoder module designed in this project decodes the incoming 32-bit instruction according to the following official ISA standards:

	(7 bit)	(5 bit)	(5 bit)	(3 bit)	[11:7] (5 bit)	[6:0] (7 bit)
R-Type	funct7	rs2	rs1	funct3	rd	opcode
I-Type	<- imm[11:0] ->	rs2 (Yok)	rs1	funct3	rd	opcode
S-Type	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B-Type	imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode
U-Type	<- imm ->	-	-	-	rd	opcode
J-Type	<- imm[20 10:1 11 19:12] ->	-	-	-	rd	opcode

Appendix B: Opcode Table

To which group the incoming instruction belongs is decoded in hardware by checking the opcode field, which consists of the 7 least significant bits (LSB) of the instruction. The basic Opcode table referenced in riscv_decoder.sv is as follows:

Instruction Group	Opcode (Binary)	Opcode (Hex)	Instruction Type Examples
R-Type ALU	0110011	0x33	ADD, SUB, AND, XOR, CLZ, CTZ, CPOP
I-Type ALU	0010011	0x13	ADDI, ANDI, SLLI, XORI
Load	0000011	0x03	LW, LH, LHU, LB, LBU
Store	0100011	0x23	SW, SH, SB
Branch	1100011	0x63	BEQ, BNE, BLT, BGE, BLTU, BGEU
JAL	1101111	0x6F	JAL
JALR	1100111	0x67	JALR
LUI	0110111	0x37	LUI
AUIPC	0010111	0x17	AUIPC

Appendix C: ALU Operations and Control Signals (alu_op_e)

The 4-bit `alu_ctrl_i` signal defined in the design's package file (`riscv_pkg.sv`) that controls the ALU and its corresponding mathematical/logical hardware operations are as follows:

Signal	Binary Code	Hardware Operation	RV32I / Custom Instruction
ALU_ADD	0000	Addition ($A + B$)	ADD, ADDI, AUIPC, Address Calculation
ALU_SUB	0001	Subtraction ($A - B$)	SUB, Branch Comparison
ALU_AND	0010	Logical AND ($A \& B$)	AND, ANDI
ALU_OR	0011	Logical OR ($A B$)	OR, ORI
ALU_XOR	0100	Logical XOR ($A \wedge B$)	XOR, XORI
ALU_SLL	0101	Shift Left Logical	SLL, SLLI
ALU_SRL	0110	Shift Right Logical	SRL, SRLI
ALU_SRA	0111	Shift Right Arithmetic	SRA, SRAI
ALU_SLT	1000	Less Than (Signed)	SLT, SLTI
ALU_SLTU	1001	Less Than (Unsigned)	SLTU, SLTUI
ALU_CLZ	1011	Count Leading Zeros	CLZ
ALU_CTZ	1100	Count Trailing Zeros	CTZ
ALU_CPOP	1101	Population Count	CPOP

Appendix D: Pipeline Registers and Signal Contents

The critical signal bundles transferred from one clock cycle to the next in the registers ensuring synchronization between stages in the 5-stage pipeline are as follows:

1. IF/ID Register
 - `if_id_pc`: Memory address of the currently fetched instruction.
 - `if_id_instr`: 32-bit raw machine code retrieved from memory.
 - `if_id_valid`: Validity bit of the instruction (set to '0' during a flush).
2. ID/EX Register
 - `id_ex_pc`: Instruction address to be used in calculations.
 - `id_ex_rs1_data` / `id_ex_rs2_data`: Raw operand values read from the register file.
 - `id_ex_imm`: Sign-extended immediate value.
 - `id_ex_rd_addr`: Destination register address (carried forward for the writeback stage).

Control Signals: `id_ex_reg_write`, `id_ex_mem_read`, `id_ex_mem_write`, `id_ex_alu_ctrl`, `id_ex_branch_type`.

3. EX/MEM Register
 - `ex_mem_alu_res`: Result calculated in the ALU or memory access address.
 - `ex_mem_rs2_data`: Data to be written to memory.
 - `ex_mem_rd_addr`: Destination register address.

Control Signals: ex_mem_reg_write, ex_mem_mem_read, ex_mem_mem_write, ex_mem_mem_width.

4. MEM/WB Register

- mem_wb_alu_res: ALU result (written to the destination if memory was not read).
- mem_wb_mem_data: Data read from external memory and formatted.
- mem_wb_rd_addr: Destination register address.

Control Signals: mem_wb_reg_write, mem_wb_wb_sel (MUX Selector).

Appendix E: Commit Forma

For verification purposes, the developed processor communicates with the testbench environment via a commit_t struct. This structure is defined in riscv_pkg.sv as follows:

```
typedef struct packed {
    logic    valid;
    logic [31:0] pc;
    logic [31:0] instr;
    logic [4:0] reg_addr;
    logic [31:0] reg_data;
    logic [31:0] mem_addr;
    logic [31:0] mem_data;
    logic    mem_wrt;
} commit_t;
```

Appendix F: Processor Datapath

The hardware roadmap followed by an instruction in the processor architecture, from being fetched from memory to its result being written back and verified, is summarized below:

